



Stack de observabilidad en k8s

Monitoreo, Trazabilidad y Análisis de Incidentes en Microservicios Fintech

Trabajo Final: Observabilidad y Confianza

El Desafío Fintech: Monitoreo vs. Observabilidad

«En finanzas, un dashboard en verde no garantiza que las transacciones se estén completando con éxito.»»

1. El Desafío (Negocio Fintech)

- Cero Tolerancia a Fallos: En transferencias de dinero, una falla silenciosa afecta fondos reales y la confianza del usuario.
- La Paradoja del Dashboard Verde: Los endpoints /health pueden responder 200 OK mientras ocurren bloqueos de base de datos downstream.
- Necesidad de Auditoría: Trazabilidad completa de punta a punta en cada débito.

2. La Solución (Observabilidad Activa)

- De "Saber si anda" a "Entender por qué falla": Ir más allá del status 200 OK tradicional.
- Diagnóstico por Telemetría: Correlación de 4 Golden Signals, Trazas (Jaeger) y Logs JSON por trace_id.
- Reducción del MTTR: Aislamiento de fallas críticas (SEV-1) en segundos.

Componentes de la Arquitectura

Transfer-api (*Gateway*):

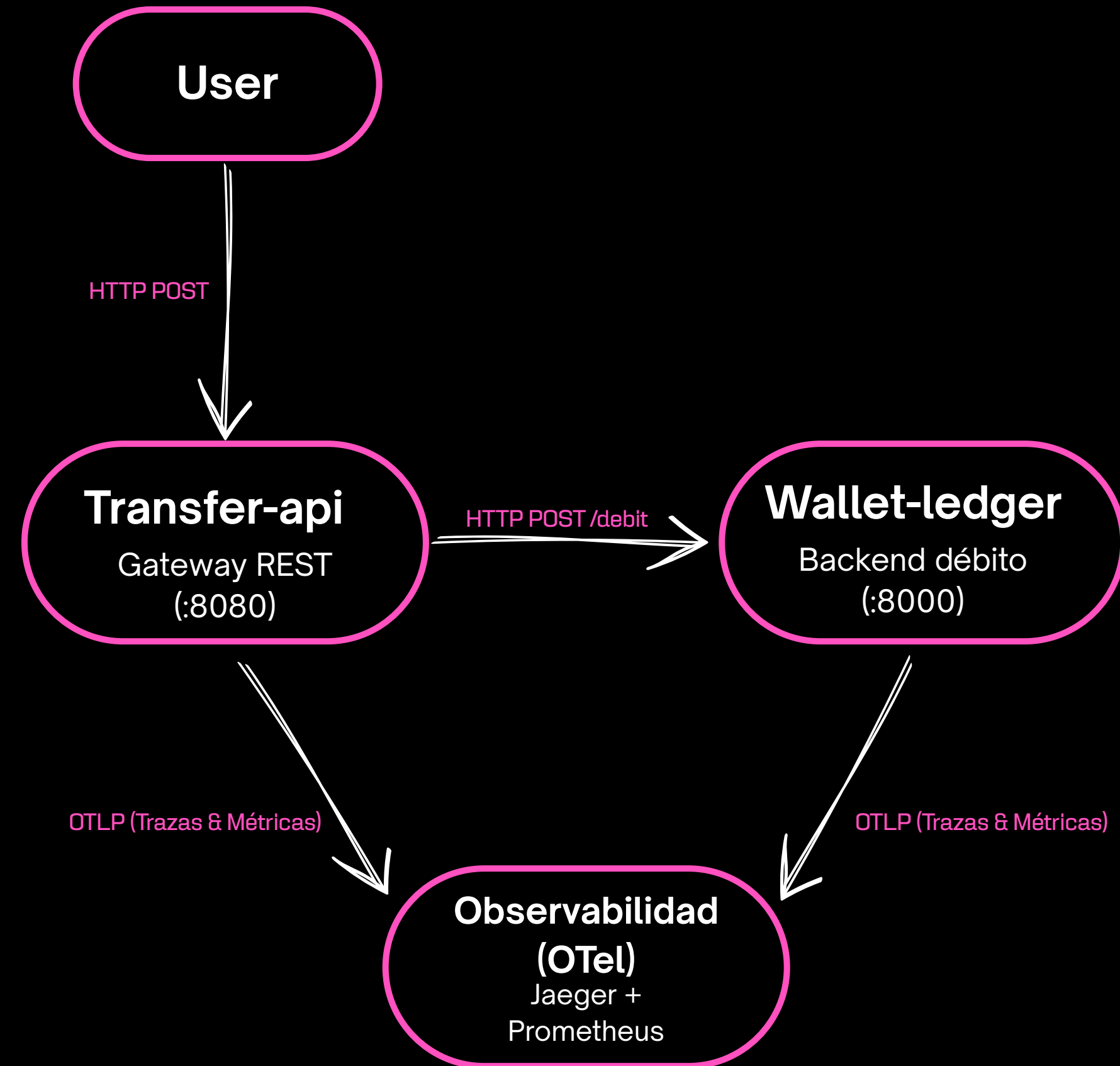
Punto de entrada HTTP REST (FastAPI). Genera el trace_id e inyecta la cabecera W3C traceparent

Wallet-ledger (*Backend*)

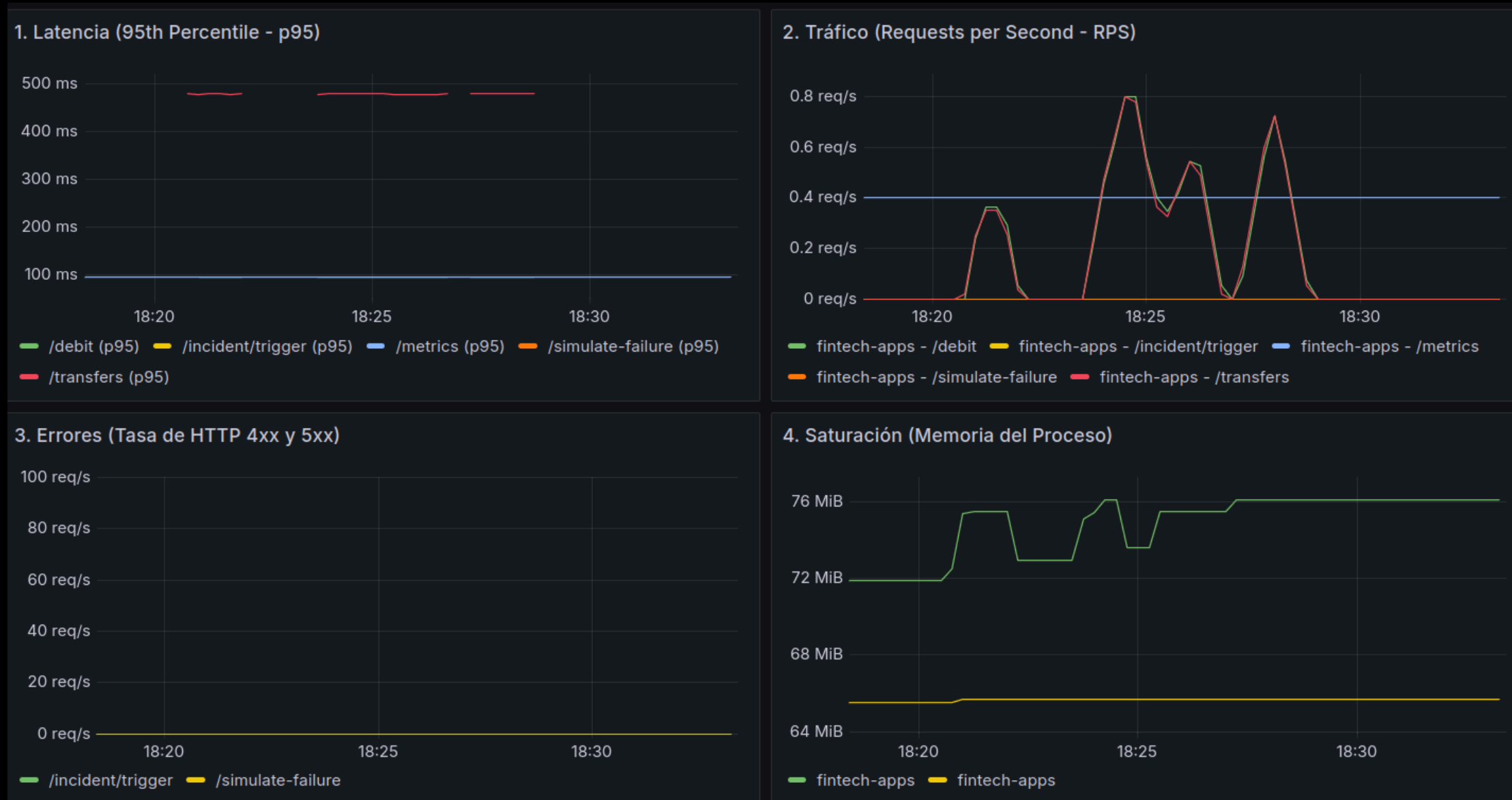
Servicio transaccional downstream. Procesa el débito y cuenta con spans manuales de negocio (validar_fondos, debitar_cuenta).

Cluster Local (*k3d*)

Aislamiento de red mediante Services internos (kube-dns) y exposición de puertos hacia el host (8080, 16686, 3000).



Diseño de Microservicios en k3d



Integración de Métricas, Trazas Distribuidas y Logs JSON

Métricas (Prometheus + Grafana)

Recolección de métricas de proceso y Dashboard con los 4 Golden Signals (Latencia, Tráfico, Errores, Saturación).

Trazas Distribuidas (OTel + Jaeger)

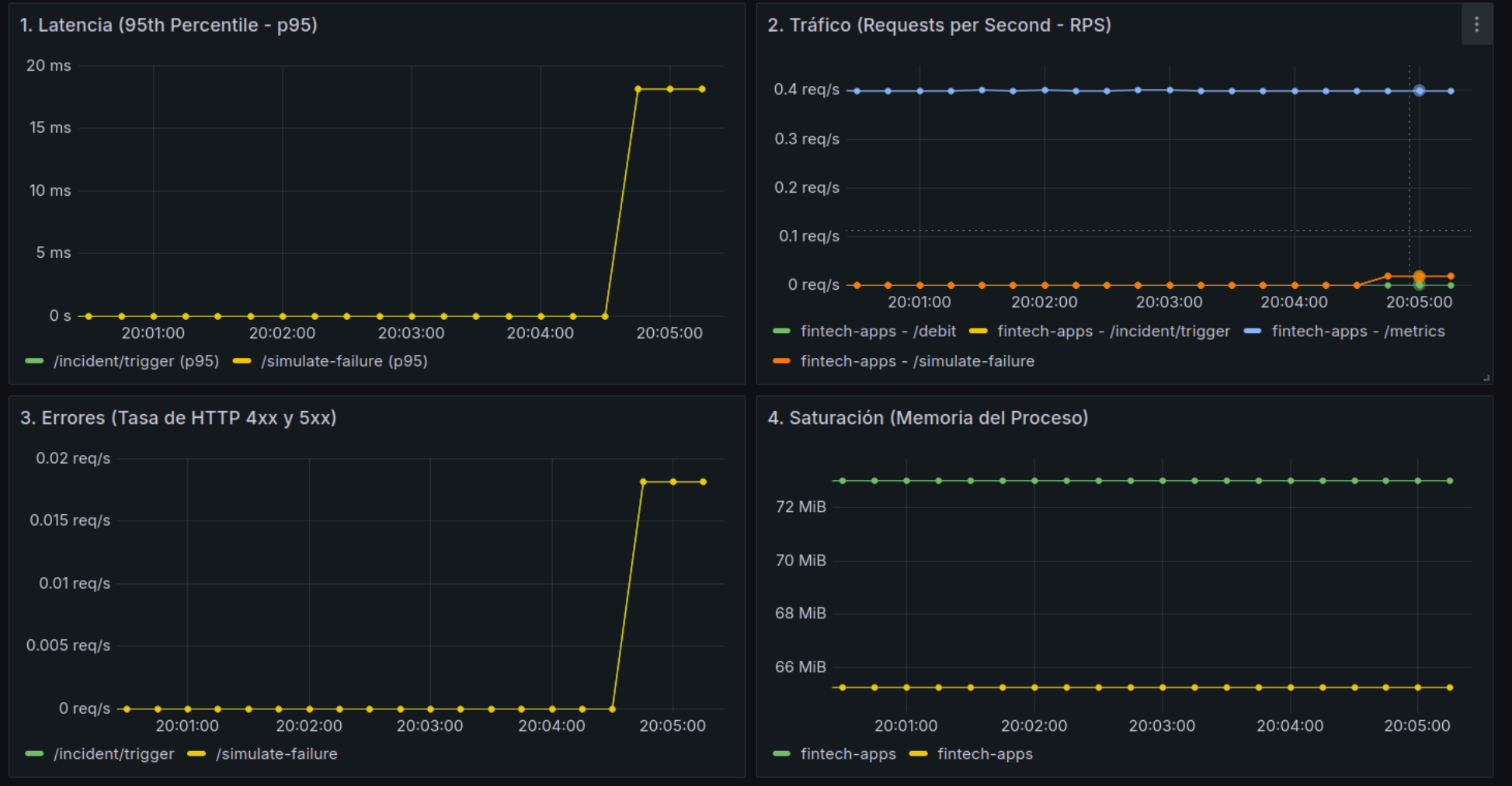
Visualización del tiempo de ejecución entre microservicios y Spans de negocio (validar_fondos, debitar_cuenta).

Logs Estructurados (structlog)

Emisión en formato JSON con inyección automática de `trace_id` y `span_id` para correlación directa.

Los 3 Pilares de la Observabilidad

Chaos Engineering: Incidente - SEV-1



El Incidente

- Simulación de bloqueo en BD (deadlock) + latencia de 13s + error HTTP 500.
- Detección por Métricas: Pico inmediato en el panel de Errores 5xx y salto latencia P95 en Grafana.

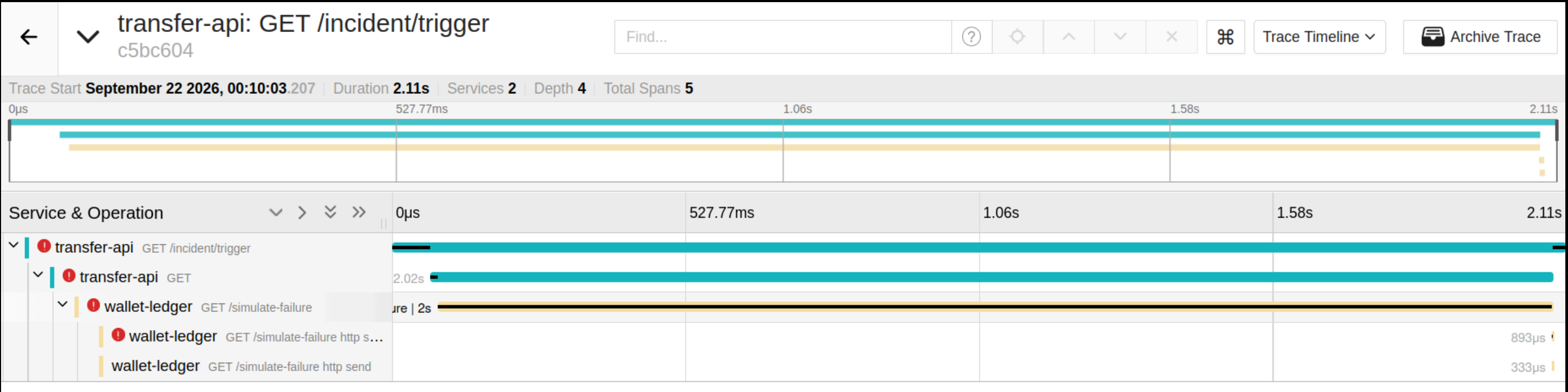
Aislamiento por Trazas

Traza marcada en rojo en Jaeger identificando la falla en wallet-ledger.

Confirmación por Logs

Filtrado de logs JSON exponiendo la excepción Database deadlock timeout.

Inyección de Falla en Cascada y Análisis de Causa Raíz (RCA)



```
{
  "event": "Simulando bloqueo de base de datos transa",
  "level": "error",
  "status_code": 500,
  "detail": "Database deadlock timeout",
  "trace_id": "6e4e8608424489d619a85297aab12b0c",
  "span_id": "9b32e7e63b9584a0"
}
```



Causa Raíz

• *Contención de Transacciones:*

Peticiones concurrentes intentando actualizar el mismo saldo al mismo milisegundo.

• *Bloqueo en BD (Deadlock):*

La base de datos abortó una de las transacciones para salvar la integridad.

• *Falla en Cascada:*

Faltaban Timeouts y Circuit Breakers en el cliente HTTP del Gateway.



Acciones Preventivas

• *Propagación de Errores (Implementado):*

Inyección de `response.raise_for_status()` para no enmascarar fallas 500 en el Gateway.

• *Patrón Circuit Breaker & Timeouts:*

Configuración de timeout máximo de 1.5s y corte de tráfico ante degradación.

• *Alerting Automatizado:*

Reglas en Prometheus para notificar a Slack si los errores 5xx superan el 2%.

Postmortem & Resiliencia

Análisis de Causa Raíz (RCA) & Plan de Acción

Conclusiones y Próximos Pasos

Beneficios del Stack y Buenas Prácticas SRE

Reducción del **MTTR** (Tiempo Medio de Resolución)

La correlación entre Métricas, Trazas y Logs permite aislar fallas complejas en segundos en lugar de horas.

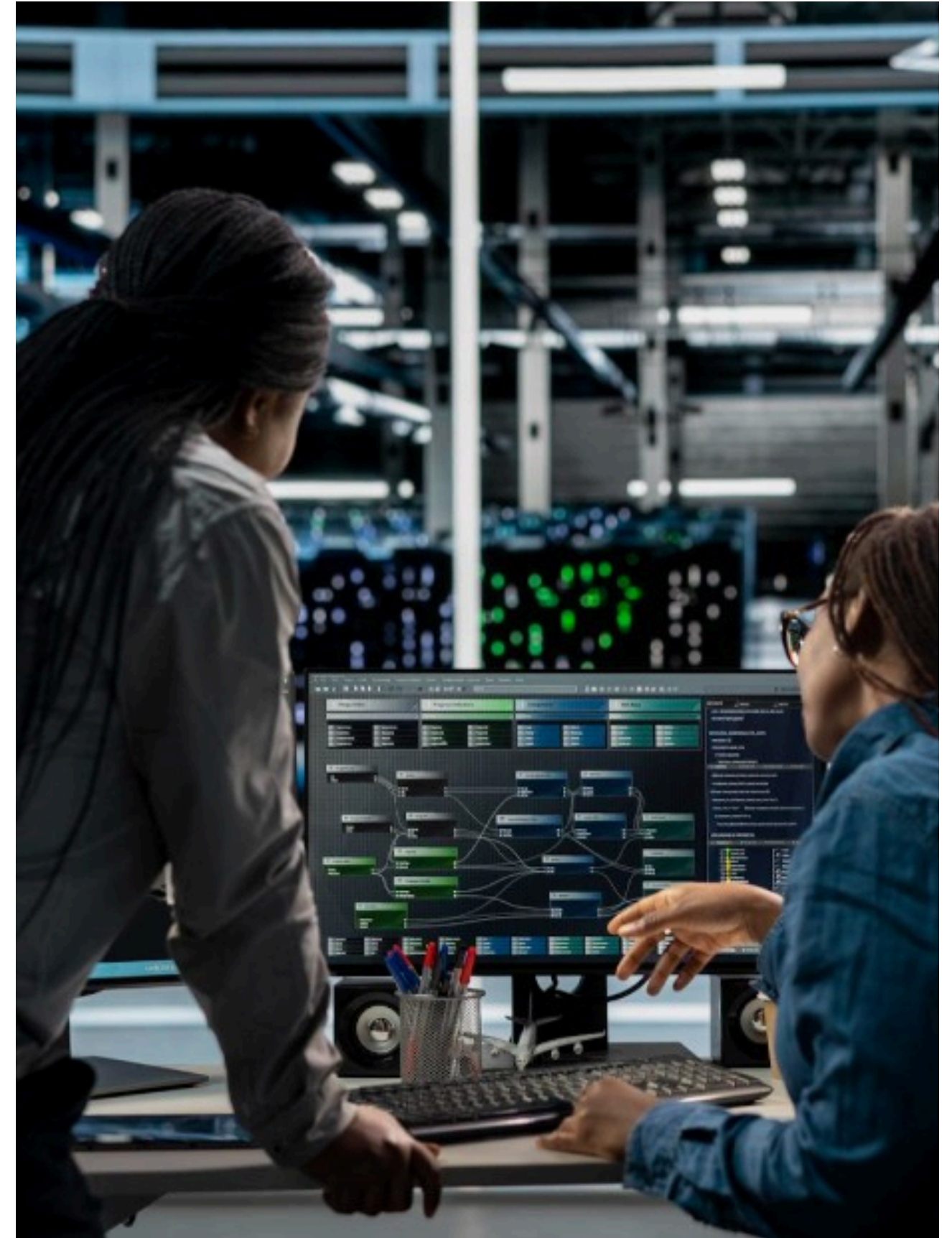
Stack **Ligero y Escalable**

Implementación completa en Kubernetes local (k3d) respetando los estándares de la industria (OpenTelemetry, W3C Trace Context, Prometheus).

Resiliencia Basada en **Evidencias**

Capacidad de simular incidentes SEV-1 y diseñar acciones preventivas (Circuit Breakers, Timeouts) respaldadas por datos.

Muchas Gracias!



www.linkedin.com/in/tomás-drago/



tomasdrawork@gmail.com